

A PROJECT REPORT ON

Sustainability Analytics & Reporting System

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aarti Lad	Exam No: B400050308
Swaraj Gosavi	Exam No: B400050407
Shubham Panchal	Exam No: B400050524
Ruchi Bhale	Exam No: B400050571



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Sustainability Analytics & Reporting System

Submitted by

Aarti Lad

Exam No: B400050308

Swaraj Gosavi

Exam No: B400050407

Shubham Panchal

Exam No: B400050524

Ruchi Bhale

Exam No: B400050571

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. S. S. Sonawane** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. S. S. Sonawane
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Sustainability Analytics & Reporting System**’.*

*We would like to take this opportunity to thank **Prof. S. S. Sonawane** for giving us all the help and guidance we needed. We are really grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions.*

In the end our special thanks to all faculty members for their whole hearted cooperation for the completion of this report. We also thank our laboratory assistants for their valuable help in the laboratory.

Aarti Lad
Swaraj Gosavi
Shubham Panchal
Ruchi Bhale
(B.E. Computer Engg.)

ABSTRACT

Software development has seen a transition from the traditional model to the agile model. In traditional software development, the testing is performed only when the development process is completed. This makes testing error-prone and time-consuming. To mitigate these challenges, the agile model was introduced, in which the development and testing occur simultaneously. Despite the advancements in software development approaches, the environmental impact continues to be a significant concern. The advancements primarily focused on improving efficiency and reducing the version delivery times, with little consideration given to their environmental impact. The energy used in software development, results in significant greenhouse gas emissions, which in turn cause global warming. The resources utilized during the software development also consume energy; hence, the resource utilization should also be optimized. By integrating sustainability metrics into software development, energy consumption, resource utilization, and other parameters can be monitored efficiently. This integration allows us to analyze the trend and make a comparison of the previous code with the newly added code to ensure that the sustainability metrics are improved or maintained with each update. A green code refiner is a tool to analyze and optimize the environmental impact of the software. Furthermore, recommendations can be provided to optimize the code, transforming it into a green code, thus promoting eco-friendly software development. This survey provides a comprehensive overview of the various approaches to green computing and the challenges associated with achieving sustainability. Through real-time monitoring and reporting of these metrics, developers and stakeholders can make informed, data-driven decisions.

KEYWORDS

Sustainability Analytics, Sustainability Metrics, Software Carbon Intensity, Carbon Emissions, Power Consumption, Sustainable Computing, Green Computing.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.3.1	Problem Definition	3
1.3.2	Objectives	3
1.4	Project Scope & Limitations	4
1.4.1	Project Scope	4
1.4.2	Limitations	4
1.5	Methodologies of Problem solving	5
2	Literature Survey	6
3	Software Requirements Specification	8
3.1	Assumptions and Dependencies	9
3.2	Functional Requirements	10
3.2.1	Data Collection and Monitoring	10
3.2.2	Reporting and Visualization	10
3.3	External Interface Requirements	10
3.3.1	User Interfaces	10
3.3.2	Hardware Interfaces	11
3.3.3	Software Interfaces	11
3.3.4	Communication Interfaces	11
3.4	Nonfunctional Requirements	12
3.4.1	Performance Requirements	12
3.4.2	Safety Requirements	12
3.4.3	Security Requirements	13
3.4.4	Software Quality Attributes	13
3.5	System Requirements	14
3.5.1	Database Requirements	14
3.5.2	Software Requirements (Platform Choice)	15

3.6	Analysis Models: SDLC Model to be applied	15
3.6.1	Phases of Agile SDLC Applied to the Project	15
4	System Design	17
4.1	System Architecture	18
4.2	Data Flow Diagrams	18
4.3	Entity Relationship Diagram	20
4.4	UML Diagrams	21
4.5	Sequence Diagram	22
5	Project Plan	23
5.1	Project Estimate	24
5.1.1	Reconciled Estimates	24
5.1.2	Project Resources	24
5.2	Risk Management	26
5.2.1	Risk Identification	26
5.2.2	Risk Analysis	27
5.2.3	Overview of Risk Mitigation, Monitoring, Management	27
5.2.4	Risk Mitigation	27
5.2.5	Risk Monitoring	28
5.2.6	Risk Management	28
5.3	Project Schedule	29
5.3.1	Project Task Set	29
5.3.2	Task Network	29
5.3.3	Timeline Chart	30
5.4	Team Organization	31
5.4.1	Team structure	31
5.4.2	Management reporting and communication	31
6	Project Implementation	32
6.1	Overview of Project Modules	33
6.2	Tools and Technologies Used	33
6.3	Algorithm Details	34
6.3.1	Algorithm 1: Metric Collection	34
6.3.2	Algorithm 2: Dashboard Integration	34
6.3.3	Algorithm 3: Natural Language Query Processing . . .	35
7	Software Testing	36
7.1	Type of Testing	37
7.2	Test cases & Test Results	37

8	Results	38
8.1	Outcomes	39
8.2	Screen Shots	39
9	Conclusions	43
9.1	Conclusions	44
9.2	Future Work	44
9.3	Applications	45
10	Appendix	46
10.1	Appendix A	47
10.1.1	Implementation Paper Details	47
10.1.2	Event Participation Details	48
10.2	Appendix B	49
11	References	50

List of Figures

4.1	Data Flow Diagram	18
4.2	Data Flow Diagram	19
4.3	Use case diagram	20
4.4	Activity Diagram	21
4.5	Sequence Diagram	22
8.1	The figure depicts a standard time-series panel available in Grafana, with data-points representing the CPU usage of different nodes. CPU usage metrics are collected for all cores present in each collector node.	40
8.2	the figure depicts the Panel Builder window of Grafana, where a desired panel can be constructed by either writing PromQL queries directly or by using their interactive query builder. Other panel metadata, such as units, title, X and Y axis labels can be set in the Panel Builder view.	40
8.3	The figure depicts the admin UI of Prometheus which is available on port 9090. The admin UI shows the available targets for metrics collection, a list of metrics being monitored and an interactive PromQL query executor.	41
8.4	The figure presents a sample from a weekly report, designed to provide a comparative analysis of key system metrics such as CPU usage and available memory. By visualizing the fluctuations in these factors over the past week, the report enables proactive identification of potential performance bottlenecks or resource constraints. This insight allows administrators to implement preventative measures, such as optimizing resource allocation or scaling infrastructure, before these issues escalate into significant system instability or service disruptions.	42
10.1	Submission Details	47
10.2	Submission Details	48

10.3 INC Certificates	48
10.4 Plagarism Report	49

CHAPTER 1

INTRODUCTION

1.1 Overview

Companies have various operational processes that release greenhouse gases into the environment. Certain laws set by the environmental protection agencies have to be followed by the companies, one of which is net zero where emissions are offset by taking certain sustainability steps. To understand what the impact of these operational processes on the environment is, a real-time tracking dashboard is needed, which will help companies take the required actions to comply with the sustainability laws. The solution presented by us provides the following. Decisions in companies are often reactive, addressing issues only after the damage occurred, and real-time monitoring systems help to take proactive decisions. Developers are increasingly prioritizing the end solution, with minimal attention given to the process of arriving at it. The growing reliance on large language models (LLMs) in corporate tools like GitHub Copilot and OpenAI models has further contributed to the environmental impact.

1.2 Motivation

The escalating energy consumption and carbon footprint associated with software applications have become a critical concern in light of increasing environmental challenges. As software permeates virtually every facet of contemporary life, spanning cloud computing and artificial intelligence to mobile applications and IoT devices, its impact on global energy demand and greenhouse gas emissions is substantial and continues to rise.

Traditional methodologies for measuring energy consumption frequently concentrate on hardware-level metrics, thereby neglecting the significant variations in energy usage and emissions that may arise from software design, architecture, and coding practices. This oversight represents a critical gap in our understanding of the true environmental impact of software.

To address this gap, there is a growing need for accurate and comprehensive tools capable of effectively measuring the energy consumption and emissions (CO₂, power consumption) of software. The availability of such tools would enable developers, researchers, and organizations to:

- Gain insights: Attain a deeper understanding of the energy profiles and emission hotspots inherent in their software.

- Optimize code: Identify areas where optimization could yield reductions in energy consumption and emissions.

- Make informed decisions: Evaluate the environmental impact of diverse software designs and technologies.

Promote sustainability: Develop and deploy software solutions that are more energy-efficient and environmentally responsible.

In this implementation, we present a novel tool that addresses this need by providing fine-grained measurement of software emissions. This tool facilitates a more thorough comprehension of software's environmental impact, paving the way for the development of more sustainable software practices.

1.3 Problem Definition and Objectives

1.3.1 Problem Definition

The escalation of energy consumption and the resultant carbon footprint of software applications constitute a significant and increasingly salient environmental concern. Software's pervasive integration into contemporary life, encompassing domains such as cloud computing, artificial intelligence, mobile applications, and IoT devices, has precipitated a substantial and continuously augmenting impact on global energy demand and greenhouse gas emissions. Traditional methodologies for quantifying energy consumption predominantly concentrate on hardware-level metrics; thereby, significant variations in energy usage and emissions attributable to software design, architecture, and coding practices are frequently neglected. This methodological focus represents a critical lacuna in the comprehensive understanding of the true environmental impact of software.

1.3.2 Objectives

The primary objective of this project is to develop a comprehensive, real-time Sustainability Analytics and Reporting System that enables fine-grained monitoring of energy consumption and carbon emissions attributable to software processes. The system aims to bridge the gap left by conventional hardware-centric measurement tools by offering software-level insights. The specific objectives of the project are as follows:

- To design and implement a real-time dashboard that visualizes key sustainability metrics such as CPU utilization, memory usage, power consumption, and carbon emissions associated with software operations.
- To quantify the environmental impact of software applications beyond hardware-level metrics by incorporating software architecture, coding practices, and runtime behavior.

- To facilitate proactive decision-making by providing organizations with timely and actionable insights into their software's energy efficiency and emission hotspots.
- To promote sustainable software development by identifying optimization opportunities that reduce resource consumption and environmental impact.
- To contribute to the broader goal of achieving net-zero emissions by equipping developers and enterprises with tools that align software development practices with environmental compliance and sustainability goals.

1.4 Project Scope & Limitations

1.4.1 Project Scope

This project aims to develop a comprehensive system for monitoring and analyzing emissions from server and virtual machine infrastructure. The system will accurately measure emission metrics, aggregate this data, and present it through an intuitive dashboard. This dashboard will cater to multiple stakeholders, providing real-time emission overviews and actionable recommendations for proactive emission reduction strategies. By offering clear insights and improvement suggestions, the system empowers informed decision-making towards a more sustainable IT environment.

1.4.2 Limitations

The proposed system exhibits certain inherent constraints. Notably, the accurate quantification of energy consumption associated with network operations and memory utilization presents a methodological challenge when relying solely on software-based instrumentation.

Furthermore, the efficacy of the system is predicated on the availability of specific hardware specifications, including the processor's Thermal Design Power (TDP) and core count. This dependency on pre-existing hardware information introduces a potential limitation in its universal applicability and necessitates careful consideration of deployment environments to ensure the reliability and validity of the generated emission metrics and subsequent recommendations.

1.5 Methodologies of Problem solving

For the emissions measurement project, the Agile methodology will be of paramount importance in facilitating iterative development, fostering collaboration, and ensuring flexibility, thereby driving continuous improvement. Practices such as sprints—discrete development cycles with clearly defined objectives, including the enhancement of data accuracy and the integration of new sensor technologies—and daily stand-up meetings will be employed to align the project team and address any impediments to progress. Furthermore, post-sprint retrospectives will enable the team to reflect upon the development process and adapt its approach, thereby optimizing the system based on acquired knowledge. A core principle of the Agile methodology is the effective management of project risks, which may include evolving regulatory requirements and unforeseen technical challenges. This methodological approach will ensure that the final emissions measurement system is subject to ongoing enhancement through continuous feedback, rigorous testing protocols, and adaptation to real-world data.

CHAPTER 2

LITERATURE SURVEY

Sustainability refers to meeting the present demands without compromising the ability of future generations to meet their needs. To quantify and benchmark environmental performance, various sustainability metrics have been proposed. The paper "Sustainability in software engineering"[5] observes the ambiguity in the definition of sustainability in software engineering and provides a detailed analysis of how the research has so far defined sustainability. Gustavo Pinto et al. address the problem of lack of knowledge of the knowledge and tools available to build sustainable solutions and, hence, explore the various engineering tools and methods in terms of enhancement methods and static analysis tools [6]. Cohen et al. discuss the growth of sustainability metrics, emphasizing their role in measuring environmental performance across different sectors [7]. In the context of data centers, Gandhi et al. introduce the Sustainable Computing Index (SCI), which measures carbon emissions per functional unit, providing a standardized metric for assessing an organization's sustainability. The SCI accounts for various sustainability costs throughout the life cycle of IT equipment, ensuring a comprehensive assessment of environmental impact [8]. Monitoring metrics such as CPU utilization, memory usage, disk I/O, power consumption, and SCI is critical for ensuring resource efficiency in AI workloads. Andrea Pazienza et al. introduce an Environmentally Sustainable Computing Framework aimed at promoting sustainable practices across industries, including design, monitoring, and refactoring [9]. With the growing adoption of Artificial Intelligence and Large Language Models, it has become crucial to assess their environmental impact and implement effective measures to minimize it. The environmental impact of training LLMs has been a subject of extensive study. Patterson et al. evaluated the carbon emissions associated with training large neural networks, highlighting that training a single LLM can consume as much energy as five cars do across their lifetimes. They emphasize the importance of optimizing model architectures and utilizing energy-efficient hardware to mitigate these impacts [10]. Similarly, on analyzing the energy efficiency and environmental impact of training LLMs, it was found that training modern LLMs can consume several megawatt-hours of electricity, significantly increasing CO₂ emissions. The study suggests that hardware and software decisions, such as selecting mixed-precision training and using Tensor Processing Units (TPUs) over Graphics Processing Units (GPUs), can improve energy efficiency by up to 30% [11]. Furthermore, Ren et al. compare the environmental impact of LLMs to human labor, revealing that while LLMs have substantial environmental impacts, their relative impacts can be dramatically lower than human labor for the same output, with human-to-LLM ratios ranging from 40 to 150 for a typical LLM [12].

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

The methodology employed in this study for measuring software energy consumption and associated carbon emissions is predicated upon the following assumptions and dependencies:

1. **Python Installation:** The system's functionality relies on the psutil Python package. A pre-existing installation of the Python interpreter on the virtual machine (VM) is therefore required to ensure the availability of necessary libraries for system resource utilization monitoring. Variations in Python versions may affect energy consumption; thus, a standardized Python environment across all VMs is recommended.
2. **Network Connectivity:** Effective operation of the monitoring infrastructure necessitates robust network connectivity between the Grafana dashboard and the Prometheus database. This connectivity is crucial for transmitting collected metrics. High-bandwidth, low-latency network infrastructure and appropriate security measures are essential to ensure reliable and secure data transfer.
3. **Processor Thermal Design Power (TDP):** Accurate calculation of heat dissipation, a key factor in estimating energy consumption, requires prior knowledge of the processor's TDP rating, typically provided by the CPU manufacturer. While TDP is a nominal value, actual power consumption may vary. Advanced power modeling techniques can improve the accuracy of energy consumption estimates.
4. **Hosting Location and Energy Attributes:** Accurate carbon emission estimates require specific geographical and energy-related data pertaining to the VM's location, including:
 - (a) **Country of VM Hosting:** The precise geographical location is required to ascertain the prevailing energy mix and associated greenhouse gas (GHG) emission factors, which vary significantly and influence the carbon intensity of electricity consumed. Geopolitical factors and international agreements may also affect GHG emissions reporting.
 - (b) **Cost per Unit of Energy:** The local cost per unit of energy is necessary for economic and operational cost estimations related to software energy consumption.
 - (c) **gCO₂ Equivalents per Unit of Energy:** The quantity of greenhouse gas emissions (in grams of CO₂ equivalents) produced per

unit of energy consumed is essential for converting energy consumption into carbon footprint estimates. Utilizing the most up-to-date and authoritative data sources is imperative for accurate calculations, as this value varies across regions and energy sources and may be subject to revisions.

These assumptions and dependencies define the operational boundaries and data requirements for the proposed measurement system. Adherence to these conditions is imperative for the accurate and reliable quantification of software energy consumption and its environmental impact.

3.2 Functional Requirements

3.2.1 Data Collection and Monitoring

The system shall collect sustainability-related data (e.g., carbon emissions, energy consumption) from various sources in real-time. It shall provide monitoring capabilities to track and analyze the data continuously, ensuring accuracy and consistency.

3.2.2 Reporting and Visualization

The system shall generate visual reports and dashboards displaying sustainability metrics. These reports shall be customizable, offering insights into emissions trends, energy usage, and performance against sustainability goals. The system shall allow users to export these reports in various formats (e.g., PDF, CSV).

3.3 External Interface Requirements

3.3.1 User Interfaces

The primary user interface for the Sustainability Analytics and Reporting System is the Grafana Dashboard. This web-based interface will provide users with:

1. A Grafana dashboard that has various charts, graphs displaying the key sustainability metrics with customizable views depending on user roles and responsibilities and allows stakeholders to analyze the trends and reduce the environmental impact.

2. A separate dashboard offers a granular and easily understandable view of key system performance indicators, focusing on the Top 5 Processes consuming the most resources (CPU, memory, and potentially power), overall Power Consumption, system Memory Usage, and system CPU Usage, where intuitive color codes enhance user understanding: green indicates metrics within the acceptable threshold, signifying a healthy state, while red immediately alerts users to metrics crossing predefined thresholds, highlighting potential inefficiencies or excessive consumption requiring attention.

3.3.2 Hardware Interfaces

1. A cloud hosted virtual machine or a self hosted machine
2. Prometheus server: Azure VM for scraping and storing sustainability metrics. Size depends on data volume/frequency.
3. Grafana server: Azure VM hosting the web UI for data visualization. Size depends on user load/dashboard complexity.
4. Python Script: Azure VM for data collection and reporting execution. Size depends on processing intensity

3.3.3 Software Interfaces

1. Operating System: Anyone among MAC OSX, Linux or Windows
2. Prometheus: The open source monitoring and alerting system
3. Grafana: The open source data visualization tool
4. Programming Language: Python

3.3.4 Communication Interfaces

The system is heavily dependent on HTTP/HTTPS for communication. The Python data collector is really a Prometheus exporter that publishes metric data via Prometheus scrape. Grafana and the reporting script, which uses Python, then access the Prometheus data using the PromQL API over HTTP to visualize the data and generate reports, respectively. Users interact with the Grafana dashboard (and a Python dashboard) by browsing the Web using HTTP/HTTPS.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- **Data Acquisition Frequency:** Collect resource utilization metrics at a frequency sufficient to capture energy consumption patterns. A minimum 1 Hz sampling rate is required, with the capability to increase it for applications with rapidly changing workloads.
- **Data Processing Latency:** Minimize the time to process the collected metrics, calculate energy consumption, and generate estimates of carbon emissions. Provide near real-time insights with a maximum 5-second latency between data acquisition and results in the Grafana dashboard.
- **Scalability:** Design the system to handle a variable number of virtual machines (VMs) and software applications without significant performance degradation. Scale horizontally by adding resources to the monitoring infrastructure to accommodate increased workloads.
- **Data Storage Capacity:** Provide sufficient and scalable storage for collected metrics and calculated results, with a minimum data retention period of 1 year.
- **System Availability:** Ensure high availability of the monitoring infrastructure, including the Grafana dashboard and Prometheus database, for continuous data collection and analysis. Achieve a minimum uptime of 99.9%.

3.4.2 Safety Requirements

- **Access Control:** Users with the right authorization have to access sensitive data and functionalities. Enforce Role-Based Access Control (RBAC).
- **Secure Authentication:** Implement strong authentication methods, such as multi-factor authentication, and ensure that all actions are authorized.
- **Versioning:** Allow tracking and recovery of prior versions of reports and thresholds if necessary.
- **Anomaly Detection:** Use of automated alerts to flag unusual patterns in emission data and to ensure timely detection of issues.

3.4.3 Security Requirements

- **Secure Storage and Access of Sustainability Data and Reports:**
To protect sensitive environmental data and maintain the integrity of sustainability reports, the system must implement strong security measures for data storage and access. Only authenticated users should be able to access this information and access should be granted based on clearly defined roles (e.g. sustainability analyst, auditor, admin).
- **Access Control for Sustainability Thresholds and Settings:**
Modifying sustainability thresholds (e.g. carbon emission limits, water use goals) can have a significant impact on operational goals and reporting. Therefore, only authorized personnel, typically system administrators or environmental compliance officers, should have the ability to change these values.

3.4.4 Software Quality Attributes

3.4.4.1 Design Qualities

The system should be designed in a modular fashion, with well-defined components and interfaces. This will allow for easier maintenance, updates, and future extensions. For example, the data acquisition, processing, and visualization components should be loosely coupled.

The architecture should support both vertical and horizontal scaling to handle increasing data volumes and user loads. This includes the ability to add more resources (e.g., CPU, memory) to existing nodes or to add more nodes to the system.

The system should be designed to facilitate testing at all levels, from unit to system testing. This includes using testable components, providing test data, and supporting test automation.

3.4.4.2 Runtime Qualities

The system should provide timely and accurate results. Key performance metrics include data acquisition frequency, data processing latency, and response time for the Grafana dashboard. These are detailed in the Performance Requirements document.

The system should operate consistently without failures. This includes ensuring data integrity, fault tolerance, and mechanisms for error handling and recovery.

The system should be accessible and operational whenever needed. High availability is crucial for continuous monitoring and data collection, as specified in the Performance Requirements (99.9

3.4.4.3 System Qualities

The system should protect sensitive data (e.g., system configurations, performance data) from unauthorized access, use, or disclosure. This includes implementing appropriate authentication, authorization, and data encryption mechanisms.

The system should be able to interact with other systems, such as data sources, analysis tools, and reporting platforms. This may involve supporting standard data formats and APIs.

3.4.4.4 User Qualities

The Grafana dashboard should be user-friendly, intuitive, and easy to navigate. It should provide clear and concise visualizations of energy consumption data, enabling users to quickly understand and analyze the information.

3.4.4.5 Non-runtime Qualities

The system should be easy to maintain, update, and modify. This includes using clear and consistent coding standards, providing adequate documentation, and designing for modularity.

3.5 System Requirements

3.5.1 Database Requirements

Prometheus is used as the time-series database for storing and analyzing the collected metrics. Prometheus is a powerful and popular choice for monitoring due to its ability to efficiently store and query large volumes of time-stamped data. Its key features, such as a flexible query language (PromQL), real-time alerting, and integration with visualization tools like Grafana, make it well-suited for capturing and providing insights into the energy consumption patterns of software applications. Prometheus's role is crucial in enabling effective performance monitoring, anomaly detection, and optimization of energy usage.

3.5.2 Software Requirements (Platform Choice)

By employing the cross-platform psutil library, our solution ensures consistent monitoring of system resources, including CPU, memory, and network, across Linux, Windows, and macOS. This facilitates the integration of Prometheus and Grafana to provide a standardized view of emissions, irrespective of the underlying platform.

3.6 Analysis Models: SDLC Model to be applied

For this project, the Agile Software Development Lifecycle (SDLC) model will be applied due to its flexibility, iterative nature, and focus on continuous improvement.

3.6.1 Phases of Agile SDLC Applied to the Project

1. **Planning and Requirements Gathering:** In the initial phase, key sustainability metrics and reporting requirements are gathered from stakeholders, including regulatory bodies, sustainability officers, and technical teams. The team will focus on understanding what data needs to be collected (e.g., carbon emissions, energy consumption), and how it will be presented in the reporting dashboards. Clear definitions of these metrics are crucial for ensuring consistency and accuracy in the data collection process.
2. **Design:** Based on the gathered requirements, the system design phase will begin. The design will involve:
 - **Architecture:** Defining how different components of the system (data collection, processing, storage, reporting) will interact, ensuring scalability and sustainability.
 - **Data Flow Design:** Creating efficient models for collecting, processing, and analyzing sustainability data.
 - **User Interface Design:** Designing intuitive dashboards and reports that stakeholders can easily interpret to make informed decisions.
3. **Development (Iterative Implementation):** The system will be developed in iterations (sprints), with each sprint focusing on a specific aspect of the project, such as the data collection module, or the reporting dashboard. Each sprint will include:

- Writing code for specific features.
 - Regular testing to ensure data integrity and system performance.
 - Incorporating sustainability analysis algorithms.
4. **Testing:** After each iteration, testing will be carried out to ensure that the system works as expected. This includes:
- Unit tests for individual components (e.g., data collection modules, reporting generation).
 - Integration tests to ensure that different modules of the system interact correctly.
 - Performance testing to ensure that the system can handle large datasets efficiently, especially as sustainability data can be extensive.
 - Validation tests to check that the data reporting meets regulatory compliance and accuracy standards.
5. **Deployment:** Once a working version of the system is ready, it will be deployed for use. Given the dynamic nature of sustainability regulations, the system will have continuous integration and deployment pipelines that allow for frequent updates, ensuring that any changes to regulations or system enhancements can be implemented quickly.
6. **Maintenance and Updates:** After deployment, the system will enter the maintenance phase, where it will be monitored for performance, and issues such as data inconsistencies or security vulnerabilities will be addressed. Additionally, updates to sustainability metrics, reporting standards, or algorithms will be incorporated to keep the system up-to-date with the latest advancements in sustainability practices.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

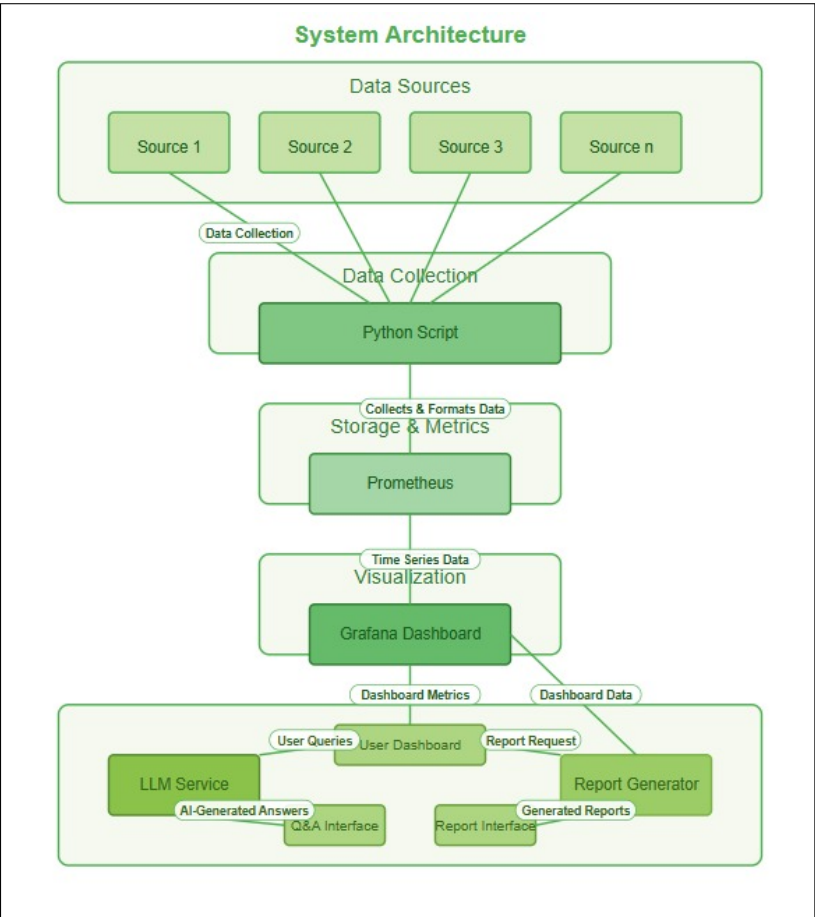


Figure 4.1: Data Flow Diagram

4.2 Data Flow Diagrams

A description of each major software function, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object oriented system) is presented.

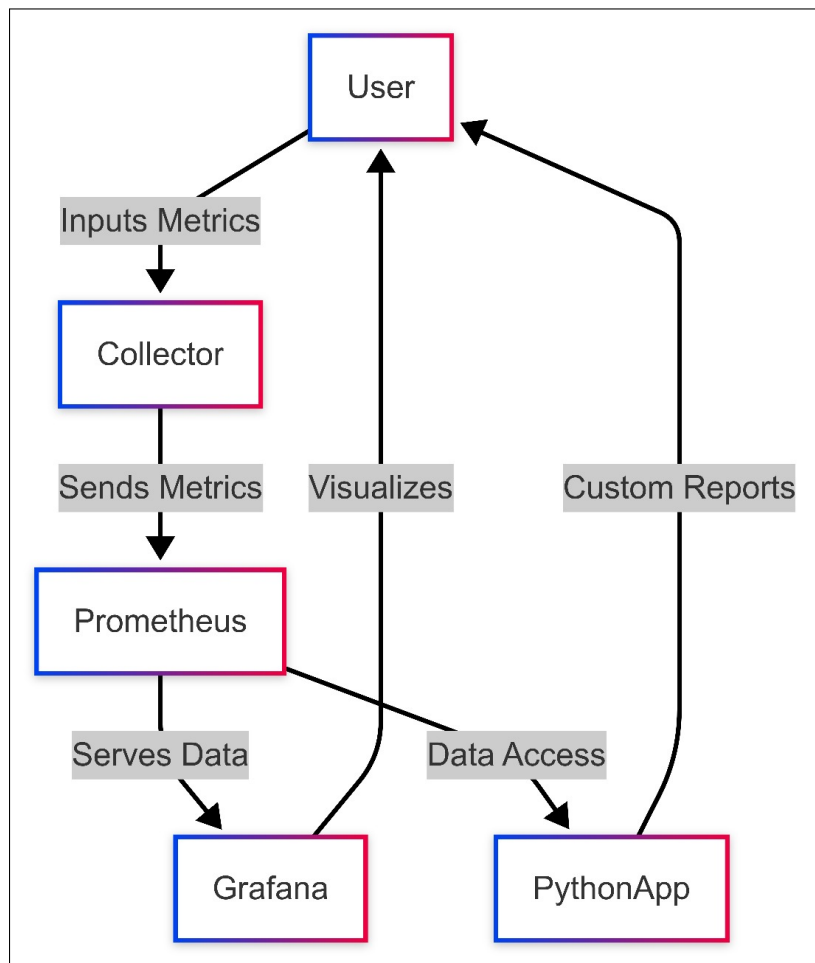


Figure 4.2: Data Flow Diagram

4.3 Entity Relationship Diagram

Here, we have shown how two different entities interact using the system architecture. Figure 4.3 illustrates the use case diagram for the Sustainability Analytics & Reporting System, showing the interactions of two main actors: the user and the admin. The user is responsible for collecting sustainability metrics such as energy consumption and system resource usage, displaying them in a real-time dashboard, generating reports, and using natural language queries to interact with the system. These features help the user monitor and analyze environmental performance efficiently.

The admin is responsible for configuring system settings, such as defining sustainability thresholds and setting alert rules to notify stakeholders when limits are exceeded. This separation of roles ensures effective monitoring, better system management, and supports informed decisions aligned with sustainable IT practices.

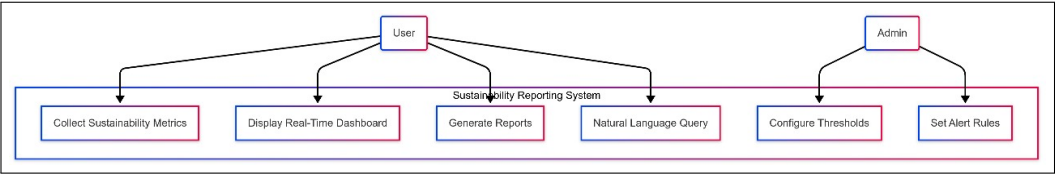


Figure 4.3: Use case diagram

4.4 UML Diagrams

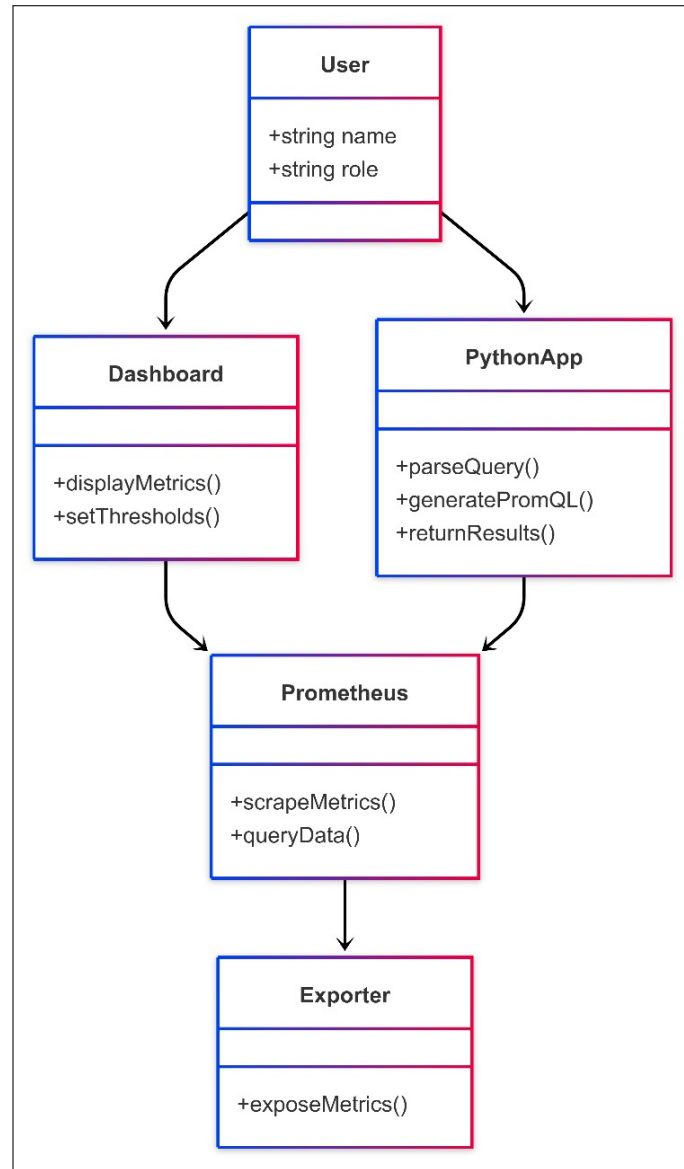


Figure 4.4: Activity Diagram

4.5 Sequence Diagram

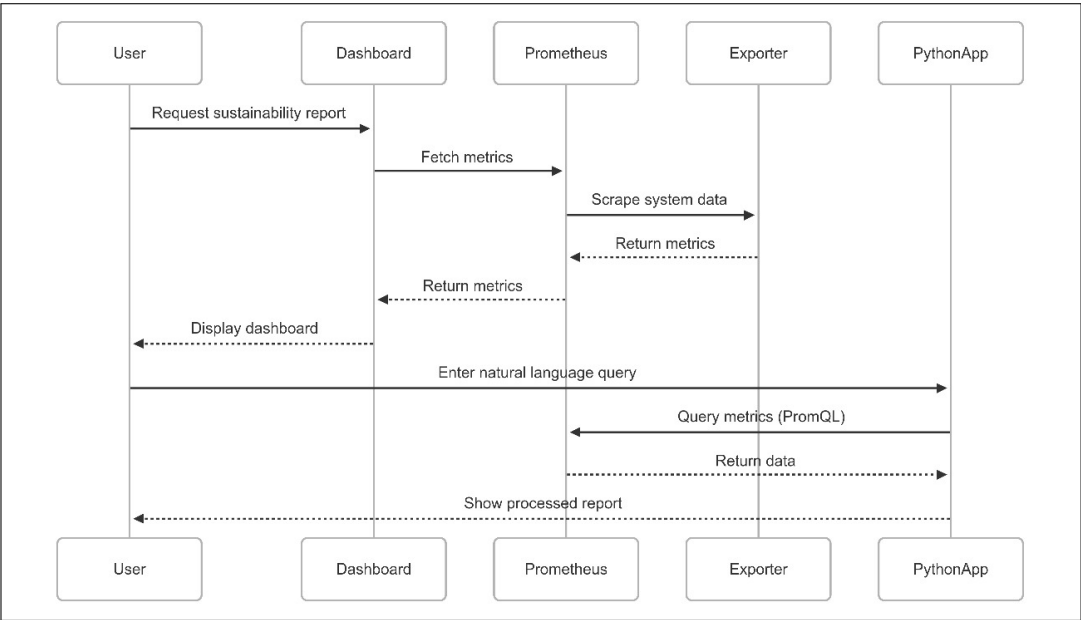


Figure 4.5: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

The reconciled estimates for the Sustainability Analytics and Reporting System were developed after comparing top-down, bottom-up, and expert-judgment-based estimation approaches. These estimates account for development, integration with monitoring tools (e.g., Prometheus, Grafana), and natural language interface setup using an LLM.

- Project Duration: 9 months (August 2024 – April 2025)
- Project cost: 100 credits + 50 credits (Azure)

5.1.2 Project Resources

The development and deployment of the **Sustainability Analytics and Reporting System** relies on a blend of cloud infrastructure, monitoring frameworks, and intelligent analytics tools to ensure scalable, real-time, and insightful sustainability reporting.

Cloud Infrastructure

- **Cloud Compute Instances (Azure VMs):** Virtual machines provisioned via Azure's free tier are used to host application backends, data processing scripts, and containerized services such as Prometheus and Grafana. These instances support continuous deployment and testing during development cycles.
- **Storage and Networking:** Azure Blob Storage is used for storing time-series data, log files, and other emissions-related metrics. Azure's virtual network features help in securing internal communication between monitoring services and data sources.

Monitoring Stack

- **Prometheus:** An open-source metrics collection and alerting toolkit used to gather, store, and query time-series data such as CPU usage, memory consumption, and emission metrics from cloud infrastructure and applications.

- **Grafana:** Integrated with Prometheus, Grafana is used to create interactive and customizable dashboards that visualize sustainability metrics, energy consumption trends, and projected carbon footprints in real time. Alerting and thresholds are configured to flag any anomalies in resource usage or emissions.

Intelligent API Integration

- **Gemini API:** The Gemini large language model API (developed by Google DeepMind) is integrated into the system to provide a natural language interface. This allows users to query sustainability data using conversational prompts such as “Show me the emissions trend for the last 30 days” or “Compare power usage across environments.” The API facilitates dynamic insights and enables users without technical expertise to explore data efficiently.

5.2 Risk Management

Effective risk management is critical to ensuring the successful delivery of the Sustainability Analytics and Reporting System. Risks were identified early during project planning and are monitored continuously throughout the project lifecycle. Each risk is assessed based on its likelihood of occurrence and its potential impact on project goals. Mitigation strategies have been defined to reduce or eliminate risks where possible.

5.2.1 Risk Identification

Technical risks:

- Inaccurate/unreliable emission measurements (CO₂, power).
- Tool incompatibility with varied software/hardware.
- Data loss/corruption during collection, storage, processing.
- Limited tool scalability for large systems.
- Integration issues with existing systems.
- Errors/biases in emission calculation models.

Data risks:

- Unavailable/inaccurate energy source and emission factor data.
- Security/privacy of collected emission data.
- Changes in emission reporting regulations.

Resource risks:

- Lack of skilled personnel in software development, energy measurement, and data analysis.
- Limited access to necessary hardware/software.

Operational risks:

- Difficulties in tool deployment/maintenance across environments.
- Poor tool usability/user adoption.

Risk	Likelihood	Impact	Overall Risk Level
Inaccurate psutil measurements	Medium	Medium	Medium
Data loss during transfer	Medium	High	High
Prometheus configuration errors	Medium	High	High
System overhead	Medium	Medium	Medium
Scalability issues	Low	High	Medium
Misinterpretation of data	Medium	Medium	Medium
Inaccurate CO2 conversion	Medium	High	High
Failure to meet regulatory requirements	Low	High	Medium/High
Data security breaches	Low	High	Medium/High

Table 5.1: Risk Assessment

5.2.2 Risk Analysis

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Effective risk management is critical to minimizing potential threats and ensuring that projects run smoothly. The process of risk mitigation, monitoring, and management involves identifying, assessing, and addressing risks throughout the project’s lifecycle.

5.2.4 Risk Mitigation

Risk mitigation in the context of the Sustainability Analytics and Reporting System involves taking preventive actions to minimize the impact of potential risks to the system’s operation and accuracy of sustainability data. Key mitigation strategies include:

- **Data Integrity and Accuracy:** Implementing quality assurance checks and validation processes to ensure that collected data is accurate and free from discrepancies. This includes using reliable sources for emissions data and ensuring that data is properly calibrated.
- **System Redundancy:** Designing fail-safes and backup systems to handle potential data loss or system downtime. This could include using cloud storage or redundant servers for continuous data collection.

- **Compliance Assurance:** Regularly reviewing and ensuring that the system adheres to relevant environmental regulations and industry standards, mitigating risks related to legal penalties or reputational damage.
- **Data Security:** Implementing encryption, access control, and secure communication protocols to safeguard against unauthorized access or data breaches.

5.2.5 Risk Monitoring

Risk monitoring in a Sustainability Analytics and Reporting System ensures that risks are continuously tracked and managed throughout the system's operation. This includes:

- **Real-Time Monitoring:** Utilizing tools like Grafana, Prometheus, and other monitoring platforms to track the performance of the sustainability metrics, ensuring that any anomalies or issues can be detected in real time.
- **Emissions Data Validation:** Regularly validating emissions data to ensure it is accurate and within expected thresholds. Any deviations can be flagged for further investigation.
- **Predictive Analytics:** Implementing machine learning algorithms that can predict potential risks, such as scaling inefficiencies, under-utilization of resources, or failure to meet sustainability targets.
- **Compliance Audits:** Continuously monitoring the system's adherence to environmental standards and regulations to prevent any non-compliance issues.

5.2.6 Risk Management

The management of risks within the Sustainability Analytics and Reporting System is an ongoing process of identifying, assessing, and addressing risks to maintain system performance and sustainability goals. The steps involved include:

1. **Risk Identification:** Recognizing and documenting potential risks related to the accuracy of sustainability metrics, system failures, data security, and regulatory compliance.

2. **Risk Assessment:** Analyzing the likelihood and impact of each identified risk. For example, assessing how a failure to accurately report emissions data could affect regulatory compliance or public trust.
3. **Risk Response Planning:** Developing actionable strategies for addressing the risks. For instance, setting up automatic alerts for anomalies in emissions data or developing contingency plans for system failures.
4. **Risk Monitoring and Control:** Continuously monitoring the system's performance, adjusting mitigation strategies as necessary, and ensuring the effectiveness of the risk management processes.

5.3 Project Schedule

5.3.1 Project Task Set

- Determine the sustainability metrics and the metrics monitoring tools and techniques.
- Set up Prometheus and Grafana for data collection and visualization.
- Develop a Python-based dashboard for custom views.
- Add a natural language interface using an LLM for ease.
- Perform testing, optimization, and documentation.
- Final deployment and project review.

5.3.2 Task Network

- Metric research → Prometheus setup → Grafana integration
- Python dashboard development → Connect to Prometheus
- LLM integration → Testing and refinement
- Final deployment → Documentation and review

5.3.3 Timeline Chart

Month	Steps to Execute
August	- Research sustainability metrics relevant to system monitoring. - Perform exploratory data analysis on energy/resource consumption. - Identify tools and methods for real-time metrics collection (e.g., Prometheus). - Setup initial Prometheus infrastructure for monitoring key system metrics.
September	- Configure Prometheus exporters for tracking sustainability metrics. - Begin collecting metrics and storing them in Prometheus. - Integrate Grafana to visualize Prometheus data in real-time. - Create sample dashboards in Grafana.
October	- Design a Python-based dashboard to provide enhanced visualizations. - Connect the dashboard to Prometheus using its HTTP API. - Begin implementation of a natural language interface using an LLM.
November	- Validate accuracy of metrics and dashboard queries. - Refine dashboard design and LLM integration.
December	- Add threshold alerts to Grafana for sustainability violations. - Finalize Python dashboard and LLM response handling.
January	- Integrate predictive analytics for sustainability trends. - Monitor system usage and improve response mechanisms.
February	- Document system architecture and workflows. - Outline best practices for sustainability-focused monitoring.
March	- Test all components for functionality and sustainability goals. - Ensure natural language queries return accurate data.
April	- Deploy final system. - Conduct final review and present outcomes and future recommendations.

Table 5.2: Timeline

5.4 Team Organization

5.4.1 Team structure

- Aarti Lad: Developed the Grafana dashboard and the natural language interface (using Gemini) for the dashboard
- Swaraj Gosavi: Developed the data collection script and deployed it on VMs for testing
- Shubham Panchal: Extended the data collection for third-party applications like Docker, Jenkins and MySQL
- Ruchi Bhale: Developed a frontend to analyze memory acquired by processes and take necessary actions, get logs from running application and display plots

5.4.2 Management reporting and communication

- A shared document repository (Google Drive) is used to track updates, reports, and deliverables.
- Internal communication is maintained via group messaging platforms (WhatsApp) for day-to-day collaboration and google meets when required.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The Sustainability Analytics and Reporting System consists of four integrated modules, each responsible for a critical function in collecting, analyzing, and communicating sustainability metrics. These modules work together to provide a seamless, real-time view of system efficiency, enabling data-driven decisions and proactive sustainability monitoring.

- **Data Collection Module:** This module leverages Prometheus, an open source monitoring solution, to collect real-time data related to system performance and resource usage. Prometheus scrapes metrics from configured exporters, such as Node Exporter, and stores them in a time-series database. These metrics form the foundation for the sustainability analysis.
- **Visualization Module:** Grafana, a powerful visualization tool, is used to present the collected metrics in a meaningful and interactive format. It allows the creation of real-time dashboards that display trends over time, support custom queries using PromQL, and enable threshold-based alerts for excessive resource usage.
- **Natural Language Query Module:** This module integrates a Large Language Model (LLM) to enable users to interact with the system using plain English. It interprets the user's intent, maps it to a PromQL query, retrieves data via the Prometheus API, and presents the results in a user-friendly format. This reduces the learning curve for technical and non-technical users alike.

6.2 Tools and Technologies Used

The system architecture incorporates a mix of monitoring, visualization, programming, and AI tools that together support an automated and insightful sustainability analytics workflow.

- **Prometheus:** An open-source time-series database and monitoring toolkit that gathers metrics from configured targets using HTTP endpoints. It supports flexible querying and alerting mechanisms.
- **Grafana:** A multi-platform analytics tool used to build dashboards and visualize time-series data from Prometheus. It supports PromQL queries and can set alerts based on defined thresholds.

- **Python:** The core language for backend development, used to build custom dashboards, automate data queries, and integrate with external services.
- **FastAPI:** Lightweight Python web frameworks used to develop RESTful APIs and web applications for hosting the custom dashboard.
- **Large Language Model (e.g, Gemini):** A transformer-based model capable of understanding and generating human language. It translates user queries into PromQL queries for Prometheus, enabling a conversational interface.
- **VS Code:** For development, experimentation, and script validation.

6.3 Algorithm Details

6.3.1 Algorithm 1: Metric Collection

Objective: Collect sustainability-related metrics like CPU usage, memory consumption, and disk activity from servers or containers.

1. Use the Bash/Batch script to install the data collector script (hosted on GitHub Gists) using the wget utility.
2. The script creates a Python virtual environment, installed required Python packages in the environment and starts executing the collector script.
3. In the Prometheus configuration, add the IP address of the node running the collector script as a target for metric aggregation.
4. Restart the Prometheus instance to apply the changes made in the configuration
5. Confirm the target is active from the Prometheus Admin UI

6.3.2 Algorithm 2: Dashboard Integration

Objective: Provide real-time visual monitoring of collected metrics.

- Setup Prometheus as a data source in Grafana
- Create panels using PromQL queries. Each panel fetches live or historical data and visualizes it using graphs, tables, gauges, etc.

- Setup Google SMTP in Grafana with Google App Passwords
- Setup thresholds for panels based on standard values

6.3.3 Algorithm 3: Natural Language Query Processing

Objective: Allow users to interact with the dashboard using plain English queries.

- The query, along with a predefined prompt, is passed to a cloud-based LLM (Gemini), instructing the LLM to generate a PromQL query for the given natural language query.
- Extract the generated PromQL query from the response of the LLM
- Pass the PromQL query to Prometheus and wait for the response
- Render the response as a table in a custom Markdown panel in Grafana

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

To ensure the quality, reliability, and accuracy of the **Sustainability Analytics and Reporting System**, various types of testing were conducted at different stages of development. These include:

- **Unit Testing**
Individual components such as data processors, API handlers, and Prometheus scrapers were tested independently to ensure correct functionality.
- **Integration Testing**
Verified seamless communication and data flow between components such as data collection modules (Prometheus), the visualization layer (Grafana), and the Gemini API.
- **System Testing**
Assessed the system as a whole in a simulated real-world environment using Azure VMs and historical emissions data to verify overall performance and feature completeness.
- **Performance Testing**
Checked the responsiveness and stability of the dashboard under load, especially when querying large datasets or generating real-time alerts.

7.2 Test cases & Test Results

Test Case	Input	Expected Output	Result
CPU load test	TDP: 65W, CPU: 0.75	Power Consumption: 48.75 W	Passed
electricity usage	Power: 48.75 W	Cost: \$0.001296	Passed
CO2 emissions	Power: 48.75 W	CO ₂ Emissions: 297.81 g	Passed

Table 7.1: Energy & Emission Testing Results

CHAPTER 8

RESULTS

8.1 Outcomes

1. The integration of Grafana provides a user-friendly interface for visualizing the collected emissions data in real-time. The dashboard displays key metrics such as electricity cost, CO2 emissions (in grams), and power consumption per operating system. This real-time feedback allows stakeholders to quickly identify energy hotspots, detect anomalies, and assess the impact of changes to the IT infrastructure.
2. A primary outcome of the project is the enhanced capacity to provide a granular, system-level perspective on energy consumption and its associated emissions. By quantifying CPU, memory, and network utilization, the system facilitates a more precise and detailed understanding of energy consumption within the IT infrastructure. This approach contrasts with conventional methodologies, which frequently rely on broader estimations or theoretical power consumption values. Furthermore, the integration with Prometheus enables the longitudinal storage and querying of these data, thereby facilitating trend analysis and historical reporting. Such transparency is of paramount importance in establishing organizational accountability for environmental impact.
3. By identifying energy inefficiencies within the IT infrastructure, the project enables organizations to reduce their energy consumption and lower their operating costs. The data can be used to optimize resource allocation, identify opportunities for virtualization or consolidation, and implement energy-saving technologies.

8.2 Screen Shots



Figure 8.1: The figure depicts a standard time-series panel available in Grafana, with data-points representing the CPU usage of different nodes. CPU usage metrics are collected for all cores present in each collector node.

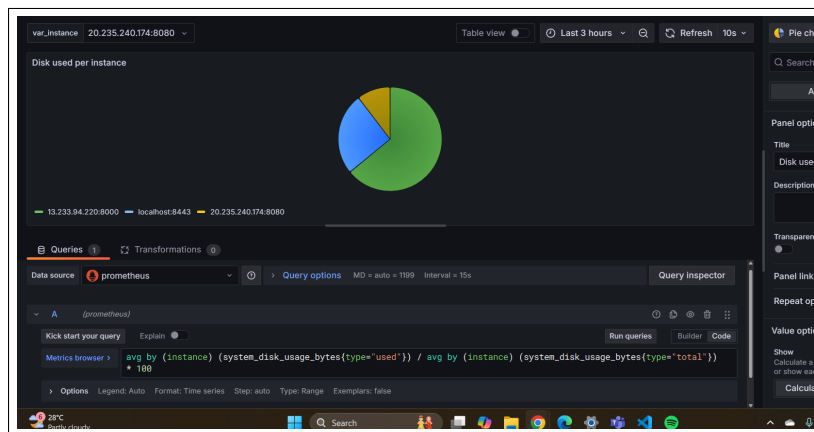


Figure 8.2: the figure depicts the Panel Builder window of Grafana, where a desired panel can be constructed by either writing PromQL queries directly or by using their interactive query builder. Other panel metadata, such as units, title, X and Y axis labels can be set in the Panel Builder view.

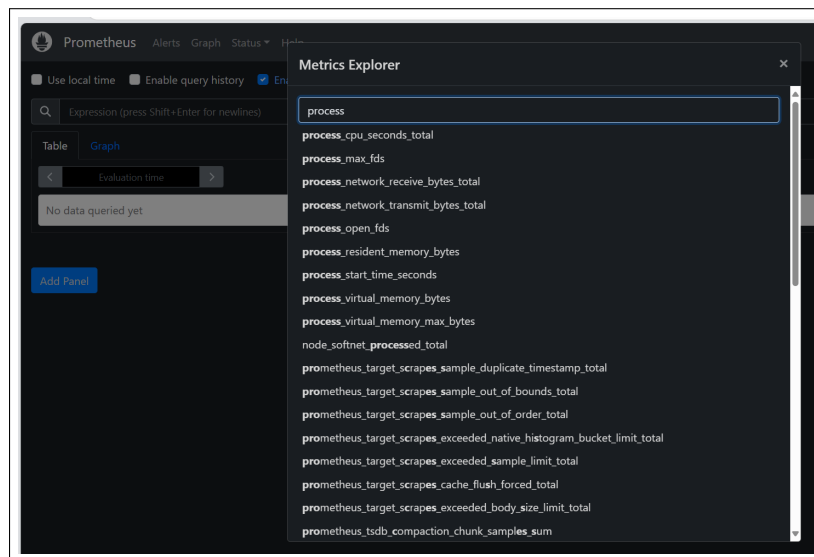


Figure 8.3: The figure depicts the admin UI of Prometheus which is available on port 9090. The admin UI shows the available targets for metrics collection, a list of metrics being monitored and an interactive PromQL query executor.

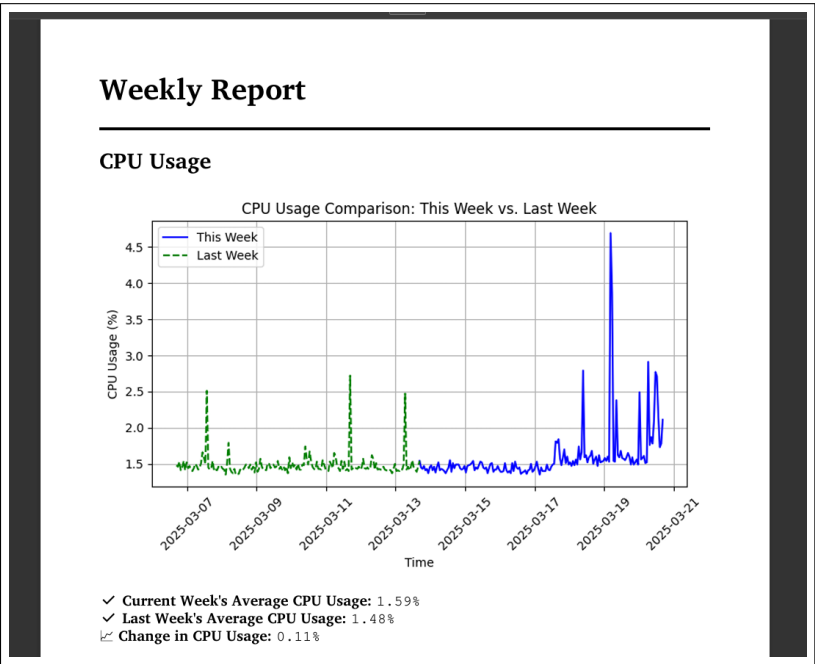


Figure 8.4: The figure presents a sample from a weekly report, designed to provide a comparative analysis of key system metrics such as CPU usage and available memory. By visualizing the fluctuations in these factors over the past week, the report enables proactive identification of potential performance bottlenecks or resource constraints. This insight allows administrators to implement preventative measures, such as optimizing resource allocation or scaling infrastructure, before these issues escalate into significant system instability or service disruptions.

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

In conclusion, this project introduces a valuable tool for organizations seeking to comprehend and manage their environmental impact. By precisely measuring and visualizing key emissions metrics, specifically CPU, memory, and network usage, the system facilitates an enhanced understanding of the energy consumption and consequent carbon footprint associated with IT infrastructure. The integration of Prometheus and Grafana provides a robust and scalable solution for real-time monitoring and longitudinal analysis. This capability empowers sustainability officers to make well-informed decisions, implement effective mitigation strategies, and ultimately ensure adherence to environmental regulatory frameworks. Future work may explore the incorporation of additional data sources, such as energy consumption at the device level, and the development of predictive models to forecast emissions and optimize resource allocation.

9.2 Future Work

The current implementation of the **Sustainability Analytics and Reporting System** serves as a foundational platform for real-time monitoring and intelligent reporting of emissions and resource usage. However, several enhancements and broader applications are envisioned for future development.

Future Work

- **Machine Learning for Forecasting:** Integration of ML models to forecast future emissions and resource usage based on historical trends, enabling proactive sustainability planning.
- **Multi-Cloud Support:** Extend the system to support and aggregate metrics from other cloud providers (e.g., AWS, GCP) for a more holistic view of infrastructure sustainability across hybrid environments.
- **Automated Recommendations:** Enable automated suggestions to optimize resource allocation, scale down underutilized instances, and improve energy efficiency—powered by usage heuristics and LLMs.
- **Mobile and Voice Access:** Develop mobile and voice-enabled interfaces to make sustainability data accessible on-the-go and hands-free, increasing system usability.

- **Policy Compliance Dashboards:** Incorporate modules to track compliance with sustainability standards such as GHG Protocol, ISO 14001, or internal ESG benchmarks.

9.3 Applications

- **Corporate ESG Reporting:** Businesses can use the system to gather and report environmental metrics for internal audits, board presentations, and ESG compliance filings.
- **Sustainable DevOps:** Integrate with CI/CD pipelines to monitor the sustainability impact of builds, deployments, and automated tests.
- **Smart Campus or Smart City Monitoring:** Adapt the system for use in monitoring energy consumption and emissions across smart buildings, university campuses, or city infrastructure.
- **Educational Tool:** Serve as an educational platform to teach students and professionals about sustainable computing, cloud efficiency, and green IT practices.
- **Research and Policy Making:** Help researchers and policymakers analyze emissions patterns and evaluate the effectiveness of green policies using real-time, granular data.

CHAPTER 10

APPENDIX

10.1 Appendix A

Paper Status: Accepted

Name of Journal: Informatica: An International Journal of Computing and Informatics

Paper Title: Sustainable Software Engineering: A Review of Green Coding Practices and Energy-Efficient Software Developments

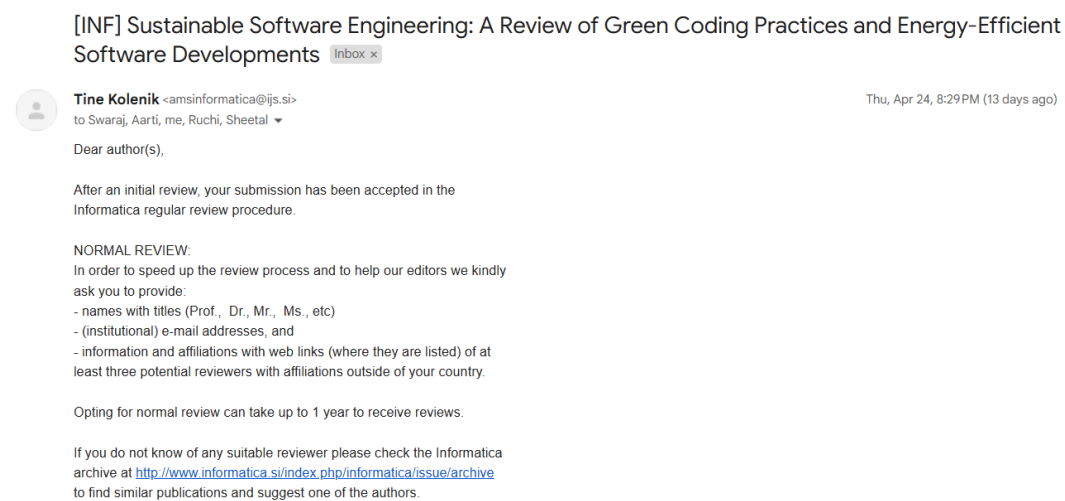


Figure 10.1: Submission Details

10.1.1 Implementation Paper Details

Paper Status: Accepted

Name of Journal: BULLETIN FOR TECHNOLOGY AND HISTORY

Paper Title: Sustainability Analytics and Reporting System for Virtualized Infrastructures

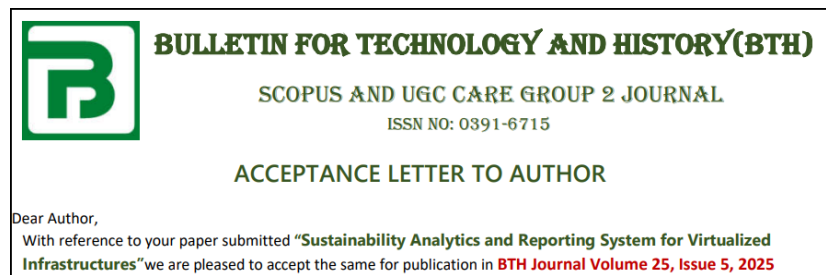
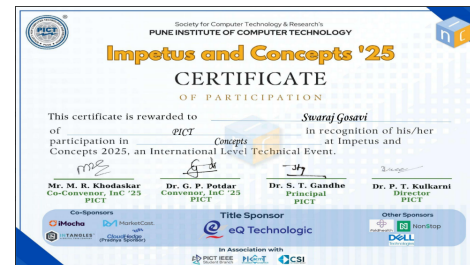


Figure 10.2: Submission Details

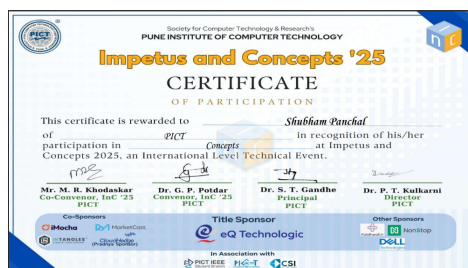
10.1.2 Event Participation Details



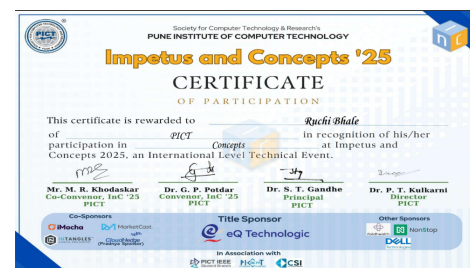
(a) Aarti INC Certificate



(b) Swaraj INC Certificate



(c) Shubham INC Certificate



(d) Ruchi INC Certificate

Figure 10.3: INC Certificates

10.2 Appendix B



Document Information	
Analyzed document	Grp_11_BE_Report_For_Plagiarism.pdf (D210788566)
Submitted	2025-05-08 11:16:00 UTC+02:00
Submitted by	
Submitter email	sssonawane@pict.edu
Similarity	1%
Analysis address	sssonawane.pict@analysis.urkund.com

Figure 10.4: Plagarism Report

CHAPTER 11

REFERENCES

- [1] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and Policy Considerations for Deep Learning in NLP. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pages 3645–3650, Florence, Italy. Association for Computational Linguistics
- [2] <https://www.iea.org/reports/data-centres-and-data-transmission-networks> International Energy Agency. (2023). Data Centres and Data Transmission Networks.
- [3] Biswas, P., Rashid, A., Biswas, A. et al. AI-driven approaches for optimizing power consumption: a comprehensive survey. *Discov Artif Intell* 4, 116 (2024). <https://doi.org/10.1007/s44163-024-00211-7>
- [4] <https://cacm.acm.org/blogcacm/the-energy-footprint-of-humans-and-large-language-models/>
- [5] Wolfram, Nina & Lago, Patricia & Osborne, Francesco. (2017). Sustainability in software engineering. 1-7. 10.23919/SustainIT.2017.8379798.
- [6] Pinto, Gustavo & Castor, Fernando. (2017). Energy efficiency: A new concern for application software developers. *Communications of the ACM*. 60. 68-75. 10.1145/3154384
- [7] Cohen, Steven Alan, Satyajit Bose, Dong Guo, Kelsie DeFrancia, Olin Berger, Brian Filiatraut, Alison C. Miller, Maureen Loman, Wen Qiu, and ChengHang Zhang. "The growth of sustainability metrics (sustainability metrics white paper series: 1 of 3)." (2014).
- [8] Anshul Gandhi, Dongyoon Lee, Zhenhua Liu, Shuai Mu, Erez Zadok, Kanad Ghose, Kartik Gopalan, Yu David Liu, Syed Rafiul Hussain, and Patrick Mcdaniel. 2023. Metrics for Sustainability in Data Centers. *SIGENERGY Energy Inform. Rev.* 3, 3 (October 2023), 40–46. <https://doi.org/10.1145/3630614.3630622>
- [9] Pazienza, A., Baselli, G., Vinci, D.C. et al. A holistic approach to environmentally sustainable computing. *Innovations Syst Softw Eng* 20, 347–371 (2024). <https://doi.org/10.1007/s11334-023-00548-9>
- [10] Patterson, David, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. "Carbon emissions and large neural network training." *arXiv preprint arXiv:2104.10350* (2021).

- [11] International Journal of Research Publication and Reviews Energy Efficiency and Environmental Impact of Training LLMs
- [12] Ren, S., Tomlinson, B., Black, R.W. et al. Reconciling the contrasting narratives on the environmental impact of large language models. *Sci Rep* 14, 26310 (2024). <https://doi.org/10.1038/s41598-024-76682-6>
- [13] Stefanos Georgiou, Maria Kechagia, Tushar Sharma, Federica Sarro, and Ying Zou. 2022. Green AI: do deep learning frameworks have different costs? In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1082–1094. <https://doi.org/10.1145/3510003.3510221>
- [14] Atadoga, Akoh & Umoga, Uchenna & Lottu, Oluwaseun & Sodiya, Enoch. (2024). Tools, techniques, and trends in sustainable software engineering: A critical review of current practices and future directions. *World Journal of Advanced Engineering Technology and Sciences*. 11. 231-239. 10.30574/wjaets.2024.11.1.0051.
- [15] Limbrunner, Nikolai. "Dynamic macro to micro scale calculation of energy consumption in CI/CD pipelines." (2023).
- [16] Aditi Sharma. "Sustainability Metrics in Software Development Practices in India." *American Journal of Computing and Engineering* ISSN 2790-5586(Online)Vol.7, Issue 4, pp 24–34, 2024
- [17] Vartziotis, Tina, Ippolyti Dellatolas, George Dasoulas, Maximilian Schmidt, Florian Schneider, Tim Hoffmann, Sotirios Kotsopoulos, and Michael Keckeisen. "Learn to Code Sustainably: An Empirical Study on LLM-based Green Code Generation." *arXiv preprint arXiv:2403.03344* (2024).
- [18] Pandey, Sunil & Singh, Kumar & Dhar, Puja & Saxena, Saurabh & Bhatnagar, Kumar. (2024). Green Computing: Importance, Approaches, and Practices. 10.1201/9781003515920-8.
- [19] <https://prometheus.io/>

LIST OF ABBREVIATIONS

Abbreviation	Illustration
TDP	Thermal Design Power
PromQL	Prometheus Query Language
ESG	Environmental, Social, and Governance
SCI	Sustainable Computing Index

LIST OF FIGURES

Figure	Illustration	Page No.
4.1	Data Flow Diagram	17
4.2	Data Flow Diagram2	18
4.3	Use Case Diagram	19
4.4	Activity Diagram	20
4.5	Sequence Diagram	21
8.1	Dashboard Image 1	38
8.2	Dashboard Image 2	39
8.3	Prometheus Admin View	39
8.4	PDF Report	40

LIST OF TABLES

Table	Illustration	Page No.
5.1	Risk Assessment	26
5.2	Timeline	29